LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 5

SINGLY LINKED LIST



Disusun oleh:

Salsadilla Hanny Azizah

109082500014

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul 5 : salsadha/PRAKTIKUMSDT

1. **Soal 1**

Singlylist.h

```
#ifndef SINGLYLIST_H_INCLUDED
#define SINGLYLIST_H_INCLUDED
#include <iostream>

#define Nil NULL
using namespace std;

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address First;
};

void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address P);
void printInfo(List L);
void insertFirst(List &L, address P);

#endif
```

Singlylist.cpp

```
// --- File: Singlylist.cpp ---
#include "Singlylist.h"

void CreateList(List &L) {
    L.First = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    if (P != Nil) {
        P->info = x;
```

```
            P->next = Nil;
        }
        return P;
    }

    void dealokasi(address P) {
        delete P;
    }

    void insertFirst(List &L, address P) {
        P->next = L.First;
        L.First = P;
    }

    void printInfo(List L) {
        address P = L.First;
        while (P != Nil) {
            cout << P->info << " ";
            P = P->next;
        }
        cout << endl;
    }
```

Source Code

```
    #include <iostream>
    #include "Singlylist.h"

    using namespace std;

    int main() {
        List L;
        address P1, P2, P3, P4, P5 = Nil;

        CreateList(L);

        P1 = alokasi(2);
        insertFirst(L, P1);

        P2 = alokasi(0);
        insertFirst(L, P2);

        P3 = alokasi(8);
        insertFirst(L, P3);

        P4 = alokasi(12);
        insertFirst(L, P4);

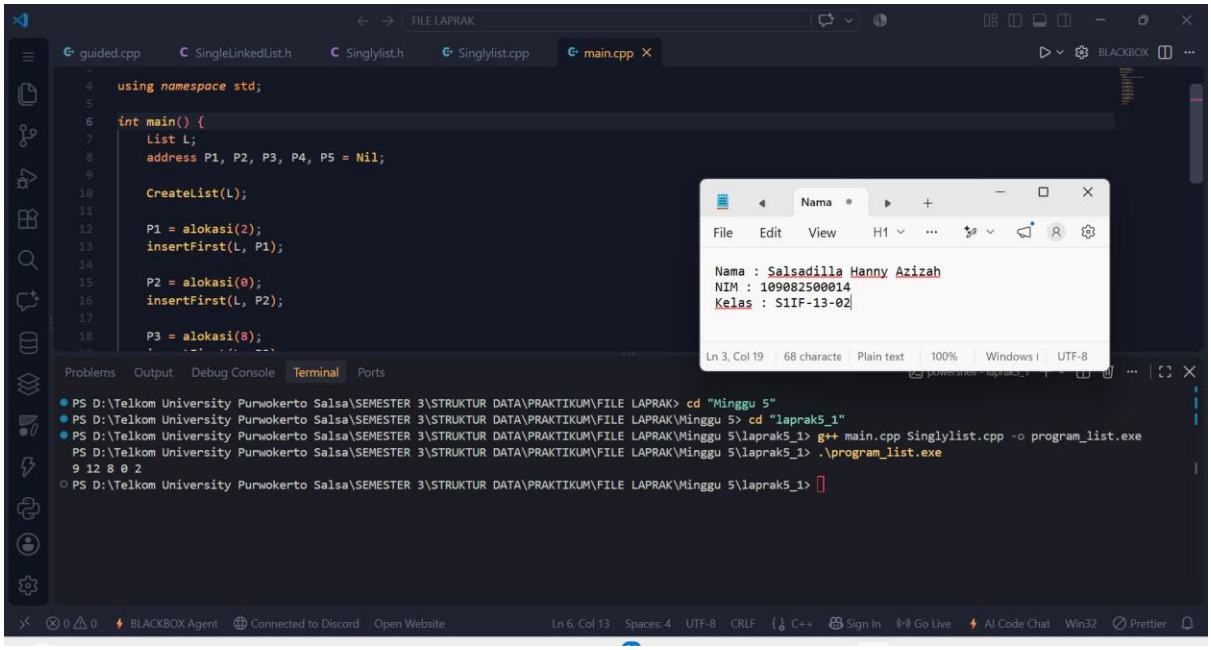
        P5 = alokasi(9);
```

```
        insertFirst(L, P5);

        printInfo(L);

        return 0;
    }
```

Screenshoot program



Deskripsi program

Program ini dibuat untuk mengimplementasikan konsep *Abstract Data Type* (ADT) pada struktur *Singly Linked List* statik dengan memecah kodenya menjadi tiga file terpisah, yaitu Singlylist.h untuk spesifikasi tipe dan prototipe, Singlylist.cpp untuk realisasi primitif fungsi, dan main.cpp sebagai program utama. Alur kerjanya berfokus pada pembentukan list kosong, pengalokasian memori untuk simpul baru, dan penyisipan data secara statis ke dalam list menggunakan prosedur insertFirst. Karena sifat penambahan elemennya selalu menumpuk dari urutan terdepan (*insert first*), data yang diinputkan mulai dari 2, 0, 8, 12, 9 otomatis akan terdorong ke belakang sehingga hasil cetakan akhirnya menjadi berurutan terbalik, yaitu 9 12 8 0 2.

Penjelasan kode program:

- **#ifndef, #define, dan #endif** Berfungsi sebagai perlindungan atau *header guards* pada file .h guna mencegah *error* akibat pendefinisian ganda apabila file tidak sengaja dipanggil berulang kali di program utama.
- **struct ElmList dan struct List** Mendeklarasikan cetakan simpul (*node*) yang memuat variabel info untuk menyimpan nilai bilangan bulat (int) dan next sebagai pointer penunjuk ke simpul berikutnya. Struktur List sendiri di sini berfungsi sebagai pengatur utama yang selalu menyimpan penunjuk awal bernama First.

- **void CreateList(List &L)** Prosedur dasar untuk menciptakan list baru dengan mengeset pointer First menjadi Nil, yang menjadi pertanda bahwa list masih kosong dan siap diisi.
- **address alokasi(infotype x)** Fungsi untuk memesan ruang memori baru melalui perintah new, lalu mengisi nilai masukan x ke bagian info, mengatur pointernya jadi Nil, dan mengembalikan alamat memori simpul segar tersebut.
- **void insertFirst(List &L, address P)** Prosedur penyisipan simpul P ke urutan paling depan list. Cara amannya, pointer next milik node P disambungkan terlebih dahulu ke elemen yang sedang menempati posisi pertama, baru setelah itu pointer utama First digeser untuk menunjuk langsung ke P agar tidak ada list yang putus atau terbuang.
- **void printInfo(List L)** Prosedur penelusuran (*traversal*) yang digunakan untuk membaca sekaligus mencetak seluruh elemen list secara sekuensial dari pangkal sampai menyentuh batas ujungnya (Nil).

2. Soal 2

Singlylist.h

```
#ifndef SINGLYLIST_H_INCLUDED
#define SINGLYLIST_H_INCLUDED
#include <iostream>

#define Nil NULL
using namespace std;

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address First;
};

void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address P);
void printInfo(List L);
void insertFirst(List &L, address P);

// Tambahan prototipe pencarian elemen
address findElm(List L, infotype x);

#endif
```

Singlylist.cpp

```
#include "Singlylist.h"

void CreateList(List &L) {
    L.First = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    if (P != Nil) {
        P->info = x;
        P->next = Nil;
    }
    return P;
}

void dealokasi(address P) {
    delete P;
}

void insertFirst(List &L, address P) {
    P->next = L.First;
    L.First = P;
}

void printInfo(List L) {
    address P = L.First;
    while (P != Nil) {
        cout << P->info << " ";
        P = P->next;
    }
    cout << endl;
}

// Tambahan realisasi pencarian elemen
address findElm(List L, infotype x) {
    address P = L.First;
    while (P != Nil && P->info != x) {
        P = P->next;
    }
    return P;
}
```

main.cpp

```
   #include <iostream>
```

```
#include "Singlylist.h"

using namespace std;

int main() {
    List L;
    address P1, P2, P3, P4, P5 = Nil;

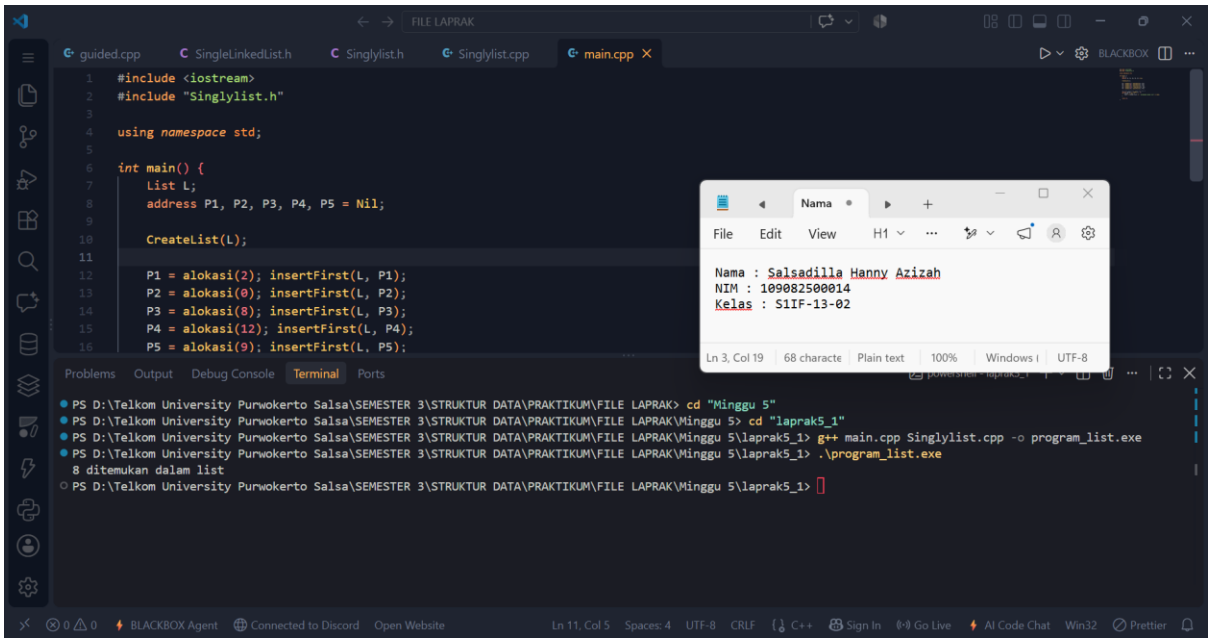
    CreateList(L);

    P1 = alokasi(2); insertFirst(L, P1);
    P2 = alokasi(0); insertFirst(L, P2);
    P3 = alokasi(8); insertFirst(L, P3);
    P4 = alokasi(12); insertFirst(L, P4);
    P5 = alokasi(9); insertFirst(L, P5);

    // Proses pencarian elemen ber-info 8
    address ketemu = findElm(L, 8);
    if (ketemu != Nil) {
        cout << ketemu->info << " ditemukan dalam list" <<
endl;
    }

    return 0;
}
```

Screenshoot program



Deskripsi program

Program ini merupakan kelanjutan modifikasi dari file ADT program sebelumnya yang ditambahkan dengan penyematan fitur pencarian letak data (*searching*). Alur program berfokus pada eksekusi fungsi findElm yang bertugas untuk mencari dan mendeteksi keberadaan simpul yang memuat nilai target berupa angka 8 di dalam rangkaian list. Program bekerja dengan cara menyisir setiap elemen secara berurutan. Begitu nilai yang dicari cocok, fungsi akan meneruskan letak alamat memorinya sehingga program utama bisa langsung menangkapnya dan mencetak validasi bahwa angka tersebut telah sukses ditemukan.

Penjelasan kode program:

- **address findElm(List L, infotype x)** Pembuatan fungsi baru dengan tipe data pengembalian berupa pointer memori (address), yang secara khusus dirancang untuk melacak dan mencari posisi simpul yang memegang nilai data x.
- **while (P != Nil && P->info != x)** Konstruksi perulangan sekuensial yang bertugas menyusuri list. Pointer penelusur P akan bergeser terus ke node sebelahnya selama belum jatuh mencapai batas akhir list (Nil) dan nilainya juga belum cocok dengan data x yang sedang diincar.
- **if (ketemu != Nil)** Blok percabangan kondisional di dalam fungsi main untuk mengecek hasil balikan dari proses pencarian. Jika alamat pointernya terbukti terisi atau tidak kosong, artinya data target sah tercantum di dalam list, maka program langsung mencetak notifikasinya dengan perintah cout

3. Soal 3

Singlylist.h

```
#ifndef SINGLYLIST_H_INCLUDED
#define SINGLYLIST_H_INCLUDED
#include <iostream>

#define Nil NULL
using namespace std;

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address First;
};

void CreateList(List &L);
address alokasi(infotype x);
```

```
    void dealokasi(address P);
    void printInfo(List L);
    void insertFirst(List &L, address P);
    address findElm(List L, infotype x);

    // Tambahan prototipe fungsi penghitung jumlah total
    int hitungTotal(List L);

    #endif
```

Singlylist.cpp

```
    #include "Singlylist.h"

    void CreateList(List &L) {
        L.First = Nil;
    }

    address alokasi(infotype x) {
        address P = new ElmList;
        if (P != Nil) {
            P->info = x;
            P->next = Nil;
        }
        return P;
    }

    void dealokasi(address P) {
        delete P;
    }

    void insertFirst(List &L, address P) {
        P->next = L.First;
        L.First = P;
    }

    void printInfo(List L) {
        address P = L.First;
        while (P != Nil) {
            cout << P->info << " ";
            P = P->next;
        }
        cout << endl;
    }

    address findElm(List L, infotype x) {
        address P = L.First;
        while (P != Nil && P->info != x) {
```

```
            P = P->next;
        }
        return P;
    }

    // Tambahan realisasi fungsi hitung total
    int hitungTotal(List L) {
        int total = 0;
        address P = L.First;
        while (P != Nil) {
            total += P->info;
            P = P->next;
        }
        return total;
    }
```

main.cpp

```
    #include <iostream>
    #include "Singlylist.h"

    using namespace std;

    int main() {
        List L;
        address P1, P2, P3, P4, P5 = Nil;

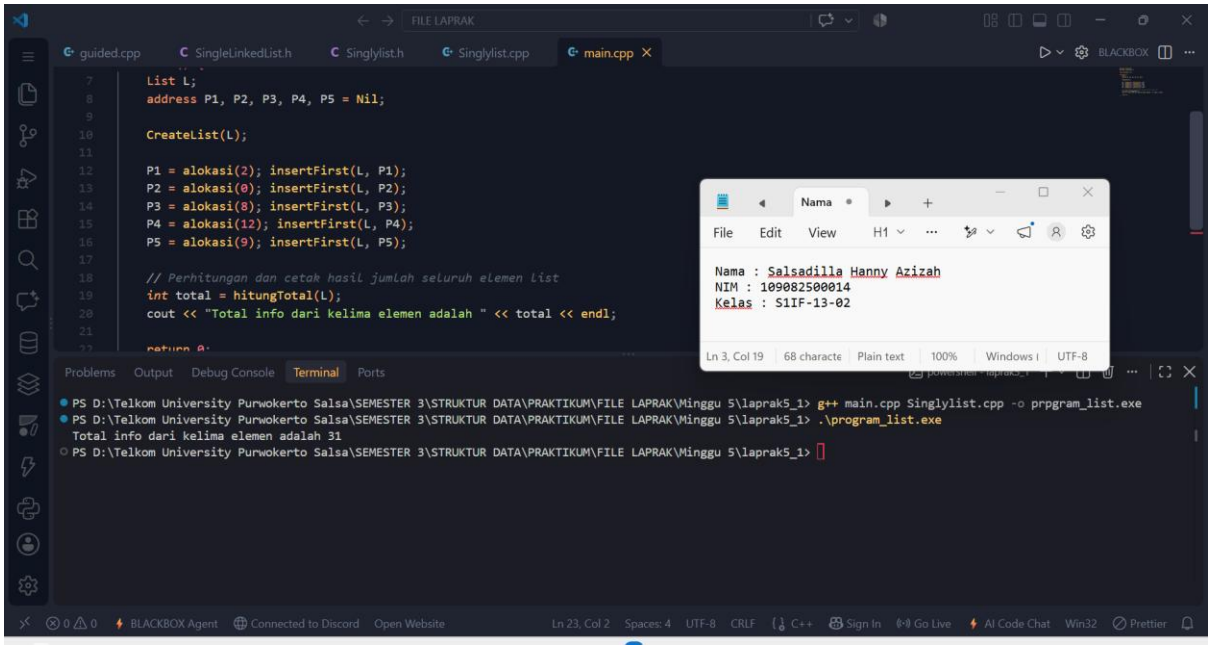
        CreateList(L);

        P1 = alokasi(2); insertFirst(L, P1);
        P2 = alokasi(0); insertFirst(L, P2);
        P3 = alokasi(8); insertFirst(L, P3);
        P4 = alokasi(12); insertFirst(L, P4);
        P5 = alokasi(9); insertFirst(L, P5);

        // Perhitungan dan cetak hasil jumlah seluruh elemen list
        int total = hitungTotal(L);
        cout << "Total info dari kelima elemen adalah " << total
    << endl;

        return 0;
    }
```

Screenshoot program



Deskripsi program

Program ini adalah pengembangan dan penyempurnaan tahap akhir dari struktur ADT Singly Linked List yang dititikberatkan pada kemampuan kalkulasi matematis terhadap isi data. Program menambahkan fitur fungsi penjumlah yang akan mengakumulasikan total keseluruhan nilai (sum) dari setiap simpul yang bernaung di dalam list. Melalui teknik penelusuran (traversal), program bergerak dari node yang terdepan sampai ujung belakang, menyomot setiap angka informasi di dalamnya, lalu menampungnya secara kumulatif. Hasil akhirnya lantas dikembalikan ke pemanggil pada fungsi utama dan dicetak rapi ke layar terminal dengan nilai total operasi berupa angka 31.

Penjelasan kode program:

- **int hitungTotal(List L)** Mendeklarasikan fungsi bantu tambahan dengan kembalian berupa integer (int) yang bertanggung jawab untuk mengekstraksi isi list sembari mengakumulasikan seluruh data elemen di dalamnya.
- **int total = 0;** Pendefinisian variabel lokal khusus di dalam bodi fungsi yang diatur awal dengan angka nol. Kehadiran variabel ini difungsikan sebagai wadah penampung hitungan sementara layaknya fungsi kalkulator.
- **total += P->info;** Perintah inti di dalam badan perulangan yang akan selalu dieksekusi terus-menerus. Perintah ini akan mengambil nilai bilangan bulat pada node yang sedang disambangi, selanjutnya disuntikkan atau ditambahkan secara kumulatif ke dalam penyimpanan variabel total.
- **return total;** Menyelesaikan operasi lalu mengembalikan lemparan nilai kalkulasi final ke main() sehingga variabel penampung output terminal siap menggunakannya dan mencetaknya secara visual dengan intervensi standar cout.

LATIHAN SOAL MODUL (GUIDED)

Link Repo Github Modul 5 : salsadha/PRAKTIKUMSDT

4. Soal 1

SingleLinkedList.h

```
#ifndef SLL_H
#define SLL_H

struct ElmList {
    int info;
    ElmList* next;
};

struct List {
    ElmList* First;
};

// Fungsi dasar
void CreateList(List &L);
ElmList* alokasi(int nilai); //alokasi untuk membuat node
baru dan mengembalikan alamat node tersebut
void dealokasi(ElmList* &P);
void printInfo(List L);

// Searching
ElmList* findElm(List L, int nilai);

// Insert
void insertFirst(List &L, int nilai);
void insertLast(List &L, int nilai);
void insertAfter(List &L, int nilaiSebelum, int nilaiBaru);

// Delete
void delFirst(List &L);
void delLast(List &L);
void delAfter(List &L, int nilaiSebelum);
void delP(List &L, int nilai);

// Operasi tambahan
void update(List &L, int nilaiLama, int nilaiBaru);
int nbList(List L);
void delAll(List &L);

#endif
```

SingleLinkedList.cpp

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;

// Membuat list kosong
void CreateList(List &L) {
    L.First = nullptr;
}

// Membuat node baru
ElmList* alokasi(int nilai) {
    ElmList* P = new ElmList;

    P->info = nilai;
    P->next = nullptr;

    return P;
}

// Membebaskan memori node
void dealokasi(ElmList* &P) {
    delete P;
    P = nullptr;
}

// Menampilkan semua isi list
void printInfo(List L) {
    ElmList* P = L.First;

    while (P != nullptr) {
        cout << P->info << " ";
        P = P->next;
    }

    cout << endl;
}

// Mencari node berdasarkan nilai
ElmList* findElm(List L, int nilai) {
    ElmList* P = L.First;

    while (P != nullptr) {
        if (P->info == nilai) {
            return P;
        }
```

```
            P = P->next;
        }

        return nullptr;
    }

    // Menambahkan node di awal
    void insertFirst(List &L, int nilai) {
        ElmList* P = alokasi(nilai);

        P->next = L.First;
        L.First = P;
    }

    // Menambahkan node di akhir
    void insertLast(List &L, int nilai) {
        ElmList* P = alokasi(nilai);

        if (L.First == nullptr) {
            L.First = P;
        } else {
            ElmList* Q = L.First;

            while (Q->next != nullptr) {
                Q = Q->next;
            }

            Q->next = P;
        }
    }

    // Menambahkan node setelah nilai tertentu
    void insertAfter(List &L, int nilaiSebelum, int nilaiBaru) {
        ElmList* Q = findElm(L, nilaiSebelum);

        if (Q == nullptr) {
            cout << "Nilai sebelumnya tidak ditemukan." << endl;
        } else {
            ElmList* P = alokasi(nilaiBaru);

            P->next = Q->next;
            Q->next = P;
        }
    }

    // Menghapus node pertama
    void delFirst(List &L) {
        if (L.First == nullptr) {
```

```
            cout << "List kosong." << endl;
        } else {
            ElmList* P = L.First;

            L.First = L.First->next;
            dealokasi(P);
        }
    }

    // Menghapus node terakhir
    void delLast(List &L) {
        if (L.First == nullptr) {
            cout << "List kosong." << endl;
        } else if (L.First->next == nullptr) {
            delFirst(L);
        } else {
            ElmList* P = L.First;

            while (P->next->next != nullptr) {
                P = P->next;
            }

            ElmList* Q = P->next;
            P->next = nullptr;
            dealokasi(Q);
        }
    }

    // Menghapus node setelah nilai tertentu
    void delAfter(List &L, int nilaiSebelum) {
        ElmList* P = findElm(L, nilaiSebelum);

        if (P == nullptr) {
            cout << "Nilai sebelumnya tidak ditemukan." << endl;
        } else if (P->next == nullptr) {
            cout << "Tidak ada node setelah nilai tersebut." <<
    endl;
        } else {
            ElmList* Q = P->next;

            P->next = Q->next;
            dealokasi(Q);
        }
    }

    // Menghapus node berdasarkan nilai
    void delP(List &L, int nilai) {
        if (L.First == nullptr) {
```

```
            cout << "List kosong." << endl;
        } else if (L.First->info == nilai) {
            delFirst(L);
        } else {
            ElmList* P = L.First;

            while (P->next != nullptr &&
                   P->next->info != nilai) {
                P = P->next;
            }

            if (P->next == nullptr) {
                cout << "Nilai tidak ditemukan." << endl;
            } else {
                ElmList* Q = P->next;

                P->next = Q->next;
                dealokasi(Q);
            }
        }
    }

    // Mengubah nilai data
    void update(List &L, int nilaiLama, int nilaiBaru) {
        ElmList* P = findElm(L, nilaiLama);

        if (P == nullptr) {
            cout << "Nilai tidak ditemukan." << endl;
        } else {
            P->info = nilaiBaru;
        }
    }

    // Menghitung jumlah node
    int nbList(List L) {
        int jumlah = 0;
        ElmList* P = L.First;

        while (P != nullptr) {
            jumlah++;
            P = P->next;
        }

        return jumlah;
    }

    // Menghapus semua node
    void delAll(List &L) {
```

```
    while (L.First != nullptr) {
        delFirst(L);
    }
}
```

main.cpp

```
#include <iostream>
#include "Singlylist.h"

using namespace std;

int main() {
    List L;
    address P1, P2, P3, P4, P5 = Nil;

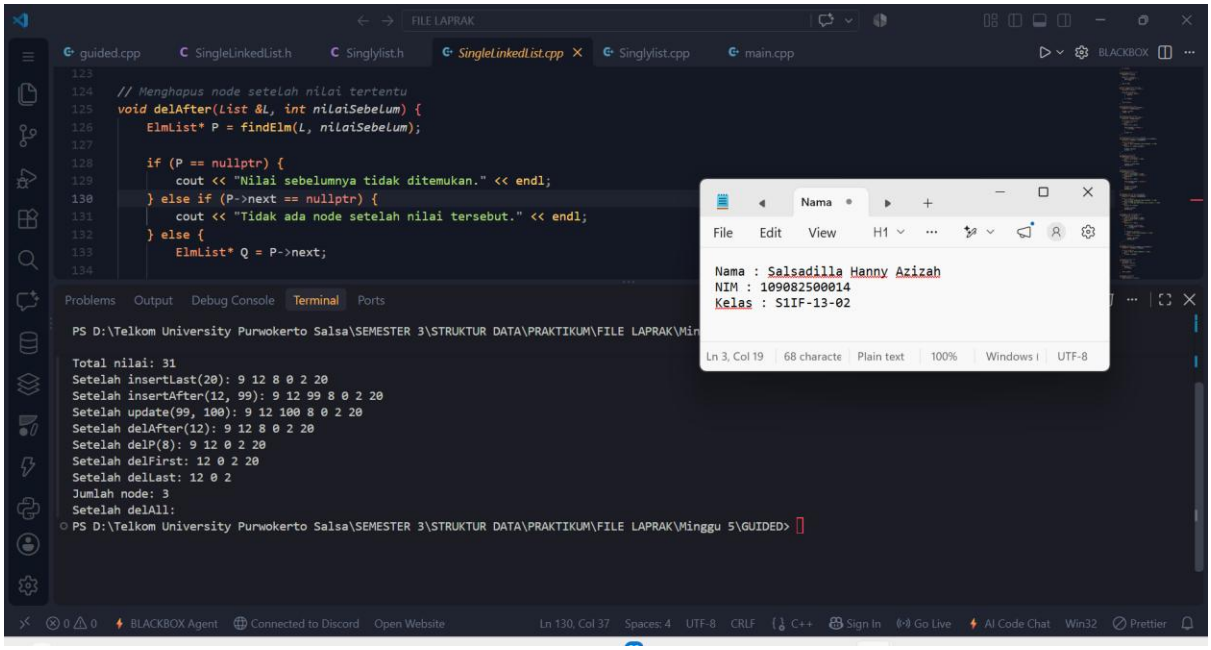
    CreateList(L);

    P1 = alokasi(2); insertFirst(L, P1);
    P2 = alokasi(0); insertFirst(L, P2);
    P3 = alokasi(8); insertFirst(L, P3);
    P4 = alokasi(12); insertFirst(L, P4);
    P5 = alokasi(9); insertFirst(L, P5);

    // Perhitungan dan cetak hasil jumlah seluruh elemen list
    int total = hitungTotal(L);
    cout << "Total info dari kelima elemen adalah " << total
<< endl;

    return 0;
}
```

Screenshoot program



Deskripsi program

Program ini dibuat untuk mengimplementasikan *Abstract Data Type* (ADT) *Singly Linked List* statik secara utuh dan komprehensif. Sesuai kaidah modular, kode dipecah ke dalam tiga file: SingleLinkedList.h (sebagai *header* spesifikasi tipe dan prototipe fungsi), SingleLinkedList.cpp (berisi realisasi seluruh operasi primitif), dan main.cpp (sebagai program penguji). Alur kerjanya mencakup eksekusi ke-12 macam operasi dasar pada *linked list*, mulai dari penyisipan data (di awal, belakang, dan tengah), pencarian letak simpul (*searching*), mengubah isi elemen (*update*), menghitung total nilai dan jumlah *node*, hingga skenario penghapusan data secara spesifik maupun menyeluruh.

Penjelasan kode program:

- **#ifndef, #define, dan #endif** Susunan *header guards* yang diletakkan di file .h untuk melindungi program dari *error* akibat pendefinisian struktur atau fungsi secara ganda.
- **struct ElmList dan struct List** Mendefinisikan tipe data bentukan. ElmList bertindak sebagai *node* (simpul) yang menyimpan nilai bilangan bulat pada variabel info serta pointer next sebagai penunjuk ke simpul selanjutnya. Sedangkan List bertugas memegang *pointer* awal bernama First.
- **alokasi(...) dan dealokasi(...)** Operasi penanganan memori. Fungsi alokasi menggunakan perintah new untuk menyiapkan memori simpul baru, sementara prosedur dealokasi menggunakan perintah delete untuk membebaskan ruang memori simpul yang sudah terhapus agar tidak boros RAM (*memory leak*).
- **Operasi Penyisipan (insertFirst, insertLast, insertAfter)** Sekumpulan prosedur untuk menambah *node* ke dalam *list*. insertFirst menambahkannya persis di paling depan, insertLast di paling belakang (ekor) dengan menelusuri dulu sampai ujung list, dan insertAfter digunakan untuk menyisipkan nilai baru tepat di sebelah kanan suatu nilai spesifik (*Prec*).

- **findElm(...) dan update(...)** Fungsi findElm melakukan pencarian elemen secara sekuensial dari depan menggunakan perulangan while hingga nilai acuan ditemukan. Prosedur update memanfaatkan fungsi pencarian ini untuk melacak nilai lama, lalu seketika menimpanya dengan nilai yang baru.
- **Operasi Penghapusan (delFirst, delLast, delAfter, delP, delAll)** Paket prosedur untuk mencabut *node*. Mencakup pencabutan *node* urutan pertama, pencabutan *node* ujung terakhir, penghapusan sesudah elemen tertentu, penghapusan dengan menargetkan nilainya langsung, hingga pengulangan *delete* untuk menghancurkan seluruh isi *list* (delAll).
- **nbList(...)** Fungsi kalkulasi yang melakukan *traversal* (penelusuran) dari *node* pertama sampai menyentuh *nullptr*. Setiap perpindahan *pointer*, variabel akumulator (jumlah) akan bertambah satu untuk menghasilkan total *node* yang aktif di dalam *list*.